

Аудит безопасности

Я провела аудит безопасности своего PHP-приложения. Ниже приведены результаты по каждому из видов уязвимостей, описание возможных рисков и применённые методы защиты. Все изменения сохранены в репозитории.

1. XSS (Cross-Site Scripting)

XSS — это тип атаки, при котором злоумышленник внедряет в веб-страницу вредоносный JavaScript-код. Этот код выполняется в браузере других пользователей, что позволяет красть кукисы, сессии, перенаправлять на фишинговые сайты или подменять содержимое страницы.

Обнаруженные проблемы

- Везде, где осуществляется вывод пользовательских данных в HTML, применяется `htmlspecialchars()`. Это эффективно предотвращает внедрение скриптов через переменные `$error`, поля форм и cookies.
- Отсутствуют флаги безопасности для сессионных и пользовательских cookies (то есть не установлены `HttpOnly`, `Secure`, `SameSite`). При возможной XSS-уязвимости злоумышленник может похитить cookie сессии.
- Не задан Content-Security-Policy (CSP), который является дополнительным уровнем защиты.

Применённые методы защиты

- Все выходные данные продолжают экранироваться через `htmlspecialchars()`. Можно сказать, это обязательное правило и это нужно оставить.
- Установлены безопасные параметры сессионных кукисов через `session_set_cookie_params()`: `HttpOnly`, `Secure` (при HTTPS) и `SameSite=Lax`.
- Добавлен заголовок CSP, ограничивающий источники скриптов.

****Установка cookie сессии с защитой в новом файле session.php**

```
ini_set('session.cookie_httponly', 1);  
ini_set('session.cookie_secure', 1); // только при HTTPS  
ini_set('session.cookie_samesite', 'Lax');
```

Php

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => null,
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Lax'
]);
session_start();
```

Замена в каждом файле `seesion_start` на указание `session.php`

```
require_once 'session.php';
```

Php

CSP-заголовок в конце файла `index.php`

```
header("Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.jsdelivr.net; style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net;");
```

Php

2. Information Disclosure (Утечка информации)

Уязвимость, при которой приложение ненамеренно отдаёт злоумышленнику чувствительные данные: путь к файлам на сервере, версии компонентов, резервные копии, исходный код, содержимое конфигурационных файлов, подробные сообщения об ошибках.

Обнаруженные проблемы

1. **login.php** — отладочная информация `$debug_info` записывается в лог с помощью `error_log()`. В неё попадают парольный хэш из БД и введенный логин. Это критическая утечка.
2. **register.php** — при ошибке PDO в выводе сообщения используется `$e->getMessage()`, раскрывающее структуру БД и запрос.
3. **admin_logout.php** — незащищённая ссылка `?emergency_reset=1` сбрасывает пароль администратора на `admin123`. И как следствие, любому, кто будет знать URL, будет доступна эта информация.

Применённые методы защиты

- Из `login.php` убран лог секретной информации `error_log($debug_info)`.
- Исключения PDO перехватываются, пользователю выдаётся сообщение: «Ошибка регистрации. Попробуйте позже», а детали записываются в лог без чувствительных данных.
- Экстренный сброс пароля полностью удалён. При необходимости восстановления применяется безопасный механизм (стандартно через email).

Обработка исключений в register.php

```
try {  
    $stmt->execute(...);  
} catch (PDOException $e) {  
    error_log('Registration failed: ' . $e->getCode()); // без деталей запроса  
    $error = 'Ошибка регистрации. Попробуйте позже.';  
}
```

Php

3. SQL Injection

Внедрение злоумышленником произвольного SQL-кода в запрос к базе данных из-за некорректной обработки входных параметров. Это позволяет читать, изменять или удалять данные БД, а иногда выполнять команды на сервере. Возникает при конкатенации строк для формирования SQL-запросов.

Как таковых уязвимостей SQL Injection не обнаружено, потому что все запросы к базе данных используют подготовленные выражения (то есть prepared statements) с плейсхолдерами или именованными параметрами. Плюсом пользовательский ввод нигде не подставляется в строку запроса напрямую.

Безопасный код проверки из БД в admin.php

```
$stmt = $db->prepare("SELECT * FROM admin_users WHERE username = ?");  
$stmt->execute([$username]);
```

Php

4. CSRF (Cross-Site Request Forgery)

CSRF — атака, при которой злоумышленник заставляет браузер аутентифицированного пользователя выполнить нежелательное действие на целевом сайте (например сменить пароль, перевести деньги, удалить данные). Используется доверие сайта к браузеру пользователя (и здесь подключается автоматическая отправка кукисов). Атака возможна, если важные действия выполняются без уникального токена, проверяющего намеренность запроса.

Обнаруженные проблемы

- Все формы приложения (то есть регистрация, вход, редактирование профиля, админ-панель) не защищены CSRF-токенами.
- В админ-панели для удаления пользователей используется GET-ссылка без токена, а значит атака хакеров может быть неизбежна.
- Хакер может заставить авторизованного пользователя выполнить нежелательные действия: сменить email, удалить аккаунт и т.д.

Применённые методы защиты

1. Генерация токена при старте сессии, хранение в `$_SESSION['csrf_token']`.
2. Вставка скрытого поля `<input type="hidden" name="csrf_token" value="...">` во все формы, изменяющие состояние (POST).
3. Проверка токена на сервере при каждой отправке формы. При несовпадении — отказ в обработке.
4. Замена удаления через GET на POST-форму с тем же токеном (в админ-панели кнопка «Удалить» передаётся через форму).

Генерация токена в начале сессии в session.php

```
if (empty($_SESSION['csrf_token'])) {  
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
}
```

Php

Вставка токена в форму index.php и edit_user.php

```
<form method="POST">  
    <input type="hidden" name="csrf_token" value="<?=$_SESSION['csrf_token'] ?>">  
    ...  
</form>
```

Html

Проверка в обработке POST

```
if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {  
    die('CSRF-ошибка: недействительный токен.');
```

```
}
```

Замена удаления в admin.php

```
// вместо <a href="?delete=..."> Php  
<form method="POST" action="admin.php" style="display:inline;">  
    <input type="hidden" name="action" value="delete">  
    <input type="hidden" name="user_id" value="<?= $user['id'] ?>">  
    <input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">  
    <button type="submit" onclick="return confirm('Удалить?')>Удалить</button>  
</form>
```

5. File Inclusion (Включение файлов)

Уязвимость, когда приложение динамически подключает файлы на основе пользовательского ввода. Злоумышленник может подставить путь к другому файлу. Это ведёт к исполнению произвольного PHP-кода или чтению серверных файлов.

Проблем с этим нет, так как путь подключения `admin_auth.php` в `edit_user.php` жёстко фиксирован и не зависит от пользовательского ввода, то есть всё безопасно. Также отсутствует динамическое включение файлов по параметрам запроса.

Если бы требовалось условное подключение, использовался бы белый список разрешённых имён и проверка реального пути, чтобы предотвратить выход за пределы директории.

6. File Upload (Загрузка файлов)

Уязвимость, когда пользователь может загрузить файл, который впоследствии выполняется сервером (PHP-скрипт) или используется для атак. Это может привести к полному захвату сервера, если злоумышленник загрузит web-shell.

Проблем с этих в коде нет, так как нет функционала загрузки файлов от пользователей.